

C Advanced Interview Questions with Answers

Medium

1. Difference between arrays and pointers?

Array

Fixed-size collection of elements

Memory allocated at declaration

Array name cannot be reassigned

Pointer

Stores an address

Can point to different locations

Pointer can be reassigned

Example:

```
int arr[5];
```

```
int *p = arr;
```

2. What is an array of pointers?

An array whose elements are pointers.

Example:

```
char *names[3] = {"Ram", "Sam", "Tom"};
```

3. What is a pointer to pointer?

A pointer that stores the address of another pointer.

Example:

```
int x = 10;
```

```
int *p = &x;
```

```
int **pp = &p;
```

4. Explain pointer to array.

A pointer that points to an entire array.

Example:

```
int arr[5];
```

```
int (*p)[5] = &arr;
```

5. What is dynamic memory allocation?

Allocating memory during program execution using heap memory.

Functions:

- malloc()
- calloc()
- realloc()
- free()

6. Why do we use malloc()?

To allocate memory dynamically at runtime.

Example:

```
int *p = (int*)malloc(5 * sizeof(int));
```

7. Difference between malloc() and calloc()?

malloc()	calloc()
Allocates memory	Allocates and initializes memory
Garbage values	Initializes to 0
Faster	Slightly slower

8. What is realloc()?

Used to resize previously allocated memory.

Example:

```
p = realloc(p, 10 * sizeof(int));
```

9. What is free()?

Releases dynamically allocated memory.

Example:

```
free(p);
```

10. What is memory leakage?

Memory allocated but never released using free().

Result:

- Wasted memory
 - Reduced performance
-

11. What are structures?

User-defined data types that group different data types.

Example:

```
struct Student {  
    int id;  
    char name[20];  
};
```

12. How do you access structure members?

Using the dot operator (.).

Example:

```
s.id = 101;
```

For pointers:

```
ptr->id
```

13. What is a nested structure?

A structure inside another structure.

Example:

```
struct Address{  
    char city[20];  
};
```

```
struct Student{  
    int id;  
    struct Address add;  
};
```

14. What is a union?

A user-defined type where all members share the same memory location.

Example:

```
union Data{  
    int i;  
    float f;  
};
```

15. Difference between structure and union?

Structure	Union
Separate memory for each member	Shared memory
Larger size	Smaller size
All members usable simultaneously	One member at a time

16. What is typedef?

Creates an alias for a data type.

Example:

```
typedef unsigned int uint;
```

17. What is an enumeration (enum)?

A user-defined type consisting of named constants.

Example:

```
enum Day {MON,TUE,WED};
```

18. What are macros?

Preprocessor definitions replaced before compilation.

Example:

```
#define PI 3.14
```

19. Difference between macro and function?

Macro	Function
Preprocessor replacement	Executed at runtime
No type checking	Type checking
Faster	Slight overhead

20. What is file handling?

The process of reading from and writing to files.

Functions:

- `fopen()`
 - `fclose()`
 - `fprintf()`
 - `fscanf()`
-

21. What are different file opening modes?

Mode Meaning

r	Read
w	Write
a	Append
r+	Read and Write
w+	Read and Write (overwrite)
a+	Read and Append

22. Difference between text files and binary files?

Text File	Binary File
Human-readable	Not human-readable
Larger size	Smaller size
Slower	Faster

23. What are file pointers?

Pointers used to access files.

Example:

```
FILE *fp;
```

24. What is conditional compilation?

Compiling specific code based on conditions.

Example:

```
#ifdef DEBUG  
printf("Debug Mode");  
#endif
```

25. What are preprocessing directives?

Instructions processed before compilation.

Examples:

```
#include  
#define  
#ifdef  
#ifndef
```

Hard**26. What is a dangling pointer?**

A pointer pointing to memory that has already been freed.

Example:

```
free(p);  
p still points to freed memory.
```

27. What is a wild pointer?

A pointer declared but not initialized.

Example:

```
int *p;
```

28. What is a void pointer?

A generic pointer capable of storing the address of any data type.

Example:

```
void *p;
```

29. What is a function pointer?

A pointer storing the address of a function.

Example:

```
int (*ptr)(int,int);
```

30. What is a self-referential structure?

A structure containing a pointer to itself.

Example:

```
struct Node{  
    int data;  
    struct Node *next;  
};
```

Used in linked lists.

31. How are linked lists implemented in C?

Using self-referential structures where each node stores:

- Data
 - Address of next node
-

32. What is a segmentation fault?

An error caused by accessing invalid memory.

Example:

```
int *p = NULL;
*p = 10;
```

33. What causes a segmentation fault?

- Dereferencing NULL pointers
 - Accessing invalid memory
 - Array out-of-bounds
 - Using freed memory
-

34. What is stack memory?

Memory used for:

- Local variables
- Function calls

Automatically managed.

35. What is heap memory?

Memory allocated dynamically using:

malloc()

calloc()

realloc()

Must be released using:

free()

36. Difference between stack and heap memory?

Stack	Heap
Automatic allocation	Manual allocation
Faster	Slower
Limited size	Larger size
Auto deallocated	Requires free()

37. Explain the memory layout of a C program.

Memory is divided into:

1. Code Segment
2. Data Segment
3. BSS Segment
4. Heap
5. Stack

38. What are Code, Data, BSS, Heap, and Stack segments?

Code Segment

- Stores program instructions

Data Segment

- Initialized global/static variables

BSS Segment

- Uninitialized global/static variables

Heap

- Dynamic memory allocation

Stack

- Function calls and local variables

39. Difference between `#include <file.h>` and `#include "file.h"`?

<>

" "

Searches system directories Searches current directory first

Used for standard headers Used for user-defined headers

40. What is the volatile keyword?

Tells the compiler that a variable's value may change unexpectedly and should not be optimized.

Example:

```
volatile int flag;
```

41. What is the restrict keyword?

Indicates that a pointer is the only way to access that memory location, allowing compiler optimizations.

Example:

```
int *restrict p;
```

42. What is token pasting (##) in macros?

Combines two tokens into one.

Example:

```
#define JOIN(a,b) a##b
```

```
JOIN(num,1)
```

becomes:

```
num1
```

43. What is stringizing (#) in macros?

Converts a macro argument into a string.

Example:

```
#define STR(x) #x
```

```
STR>Hello)
```

becomes:

```
"Hello"
```

44. What is undefined behavior in C?

Behavior for which the C standard does not define an outcome.

Example:

```
int x = 5;
```

```
x = x++;
```

Result is unpredictable.

45. What is memory alignment?

Arranging data in memory according to hardware requirements for faster access.

46. What is structure padding?

Extra bytes added by the compiler to align structure members.

Example:

```
struct Test{  
    char c;  
    int i;  
};
```

The compiler may insert padding between c and i.

47. Difference between shallow copy and deep copy?

Shallow Copy	Deep Copy
---------------------	------------------

Copies addresses	Copies actual data
------------------	--------------------

Shared memory	Independent memory
---------------	--------------------

Faster	Safer
--------	-------

48. What are command-line macros?

Macros passed during compilation.

Example:

```
gcc -DDEBUG test.c  
#ifdef DEBUG  
printf("Debug");  
#endif
```

49. How does a compiler process a C program?

The compiler converts source code into executable machine code through multiple stages.

50. Explain the stages of compilation.

1. Preprocessing

- Handles #include, #define

2. Compilation

- Converts C code to assembly code

3. Assembly

- Converts assembly code to object code

4. Linking

- Combines object files and libraries into an executable

Flow:

Source Code (.c)



Preprocessor



Compiler



Assembler



Object File (.o)



Linker



Executable File